CE-355 Lab 4
Mandelbrot Set Explorer

1. Introduction

The Mandelbrot set is one of the most famous images in mathematics. It comes from a rule that takes only one line to write, yet it produces endless detail: zoom into its edge and new spirals, filaments and miniature copies of the whole set keep appearing. In this lab, you will build an FPGA design that computes the Mandelbrot set in real time and displays it on an HDMI monitor. The push buttons pan the view, a switch zooms in and out, and every frame is computed from scratch as the screen is drawn.

The challenge is speed. An HDMI video signal has to deliver the whole image frame 60 times every second, and this design has no memory to store a finished picture in, so it must compute each pixel at the moment it is sent to the monitor. That leaves time for only a few clock cycles per pixel, but finding the color of a single pixel means repeating the same small calculation hundreds of times. A circuit that works on one pixel at a time would fall far behind the display. The design solves this using parallel processing: it builds **N identical computing engines** and runs them in parallel, each working on a different pixel.

This lab introduces five new VHDL and RTL concepts:

- **Signed and unsigned arithmetic:** the signed and unsigned types from the ieee.numeric_std package, which give a std_logic_vector a numeric meaning so that you can add, subtract, multiply and compare it.
- **Resizing:** the resize function, which widens or narrows a number so that a sum or product has enough bits and does not overflow.
- **Fixed-point numbers:** a way to represent fractions using only integer hardware, by treating the low bits of a number as the fraction.
- **Parallel computing with for-generate:** using a for-generate statement to build many identical computing engines that work on different pixels at the same time.
- **Writing binary files from a testbench:** using VHDL file I/O to save simulation results as image files that you can open and look at, instead of reading waveforms.

It also introduces two new parts of an FPGA system:

- **HDMI video output:** generating the timing for a standard video mode and sending pixels to a monitor through the board's HDMI transmitter.
- **PLL clocking:** using a **PLL** (phase-locked loop) to generate a faster system clock from the board's oscillator, and writing **timing constraints** so that Quartus can check the design at that clock speed.

Starter VHDL files are provided with **TODO sections** that you will complete. The HDMI output, the viewport (pan and zoom) and the top-level system with its PLL are provided for you. You will then simulate the design, view the simulated frames as images, and synthesize, program and test it on the DE25-Standard.

By completing this lab, you will:

- Use signed and unsigned from numeric_std to perform arithmetic on std_logic_vector signals.
- Use resize to control the width of a result and prevent overflow.
- Represent fractional values with fixed-point numbers, and rescale the result of a fixed-point multiply.
- Implement an iterative algorithm as a clocked process that performs one step per clock cycle.
- Use a for-generate statement to distribute work across a bank of parallel engines.
- Explain how an HDMI video signal is timed, and why the design must keep up with it.
- Explain why the design needs a PLL, and write timing constraints for a design that uses one.
- Write a binary file (a BMP image) from a VHDL testbench to check simulation results.
- Simulate, program and test a real-time FPGA video system.

2. Background: The Mandelbrot Set

Recommended viewing before you start: **The Mandelbrot Set**, a short video introduction to what the set is and why it looks the way it does:

https://www.youtube.com/watch?v=FFftmWSzgmk

2.1 The Rule

Pick a point on a plane, with horizontal coordinate **cx** and vertical coordinate **cy**. Start a second point **(x, y)** at the origin, (0, 0), and move it over and over with this rule:

$$x_{\text{new}} = x^2 - y^2 + c_x$$
$$y_{\text{new}} = 2xy + c_y$$

Each time the rule is applied is one **iteration**. After each iteration, measure how far (x, y) is from the origin. Depending on the starting point (cx, cy), one of two things happens:

- **The distance stays small forever.** (x, y) keeps moving, but never gets farther than 2 from the origin. The point (cx, cy) is **in** the Mandelbrot set.
- **The distance grows without limit.** (x, y) gets farther from the origin with every iteration, heading off to infinity. We say it **runs away**, and the point (cx, cy) is **outside** the set.

The hardware cannot wait forever to see which case it is, so it uses a simple test: **a point has escaped as soon as (x, y) is more than 2 away from the origin.** Once that happens, the point is

guaranteed to run away, so there is no need to keep going. The number of iterations it took to escape is the point's **escape count**.

The escape count is what sets each pixel's color. Every possible count, from 1 up to the maximum, is assigned its own color: for example, points that escape in 1 or 2 iterations might be dark blue, points that take 20 might be white, and points that take 60 might be orange. Points that never escape are painted black. So the colors in a Mandelbrot picture are not part of the math. They are a map of **how many iterations** each point took to escape.

For example:

Iteration	**(cx, cy) = (–1, 0): x is**	**Distance**	**(cx, cy) = (0.5, 0): x is**	**Distance**
start	0	0	0	0
1	−1	1	0.5	0.5
2	0	0	0.75	0.75
3	−1	1	1.06	1.06
4	0	0	1.63	1.63
5	−1	1	3.15	**3.15 > 2: escaped**
…	bounces between −1 and 0 forever	never more than 1	10.4, 110, …	keeps growing

Note that y stays 0 for both points, because cy = 0, so the rule is just $x_{new} = x \cdot x + cx$. The point (−1, 0) never gets more than 1 away from the origin, so it is **in** the set. The point (0.5, 0) creeps outward slowly, then passes a distance of 2 at iteration 5, so it is **outside** the set, with an escape count of 5.

To draw the set, treat every pixel on the screen as one starting point (cx, cy), and run the rule for each pixel.

2.2 Where the Rule Comes From: Complex Numbers

The rule is normally written with **complex numbers**. A complex number is a pair of ordinary numbers, a **real part** and an **imaginary part**, written as

$$z = x + y\,i$$
$$c = c_x + c_y\,i$$

where x and cx are the real parts, and y and cy are the imaginary parts.

Here ***i*** is the imaginary unit, defined by $i \times i = -1$, and it only marks which number is the imaginary part. So $z = x + y \cdot i$ is just the point (x, y), and $c = cx + cy \cdot i$ is the starting pixel (cx, cy). Plotting complex numbers this way, with the real part on the horizontal axis and the imaginary part on the vertical axis, is called the **complex plane**.

With complex numbers, the whole rule is one line:

$$z \leftarrow z^2 + c$$

starting from z = 0.

Multiplying out z × z and using i × i = −1 gives the two equations:

$$\begin{aligned} z^2 &= (x + y\,i)(x + y\,i) \\ &= x^2 + 2xy\,i + y^2\,i^2 \\ &= (x^2 - y^2) + (2xy)\,i \end{aligned}$$

The real part of z^2 is $x^2 - y^2$, and the imaginary part is 2xy.

Adding c adds cx to the real part and cy to the imaginary part. VHDL has no complex-number type, and the design does not need one: it stores the real part and the imaginary part as two separate signed numbers, and i never appears in the hardware.

2.3 The Escape Test

The distance of (x, y) from the origin is $\sqrt{(x^2 + y^2)}$. Why is 2 the cutoff? Squaring a number makes it bigger when the number is larger than 1, and much bigger when it is larger than 2. Once (x, y) is more than 2 from the origin, the squaring in the rule pushes it out faster than adding c can pull it back, so the distance grows on every iteration from then on. (This works because every point of interest has c no farther than 2 from the origin. A point with c farther than 2 escapes on its first iteration anyway.) So a point has **escaped** as soon as its distance is greater than 2.

To avoid computing a square root in hardware, square both sides:

$$\sqrt{x^2 + y^2} > 2 \quad \Leftrightarrow \quad x^2 + y^2 > 4$$

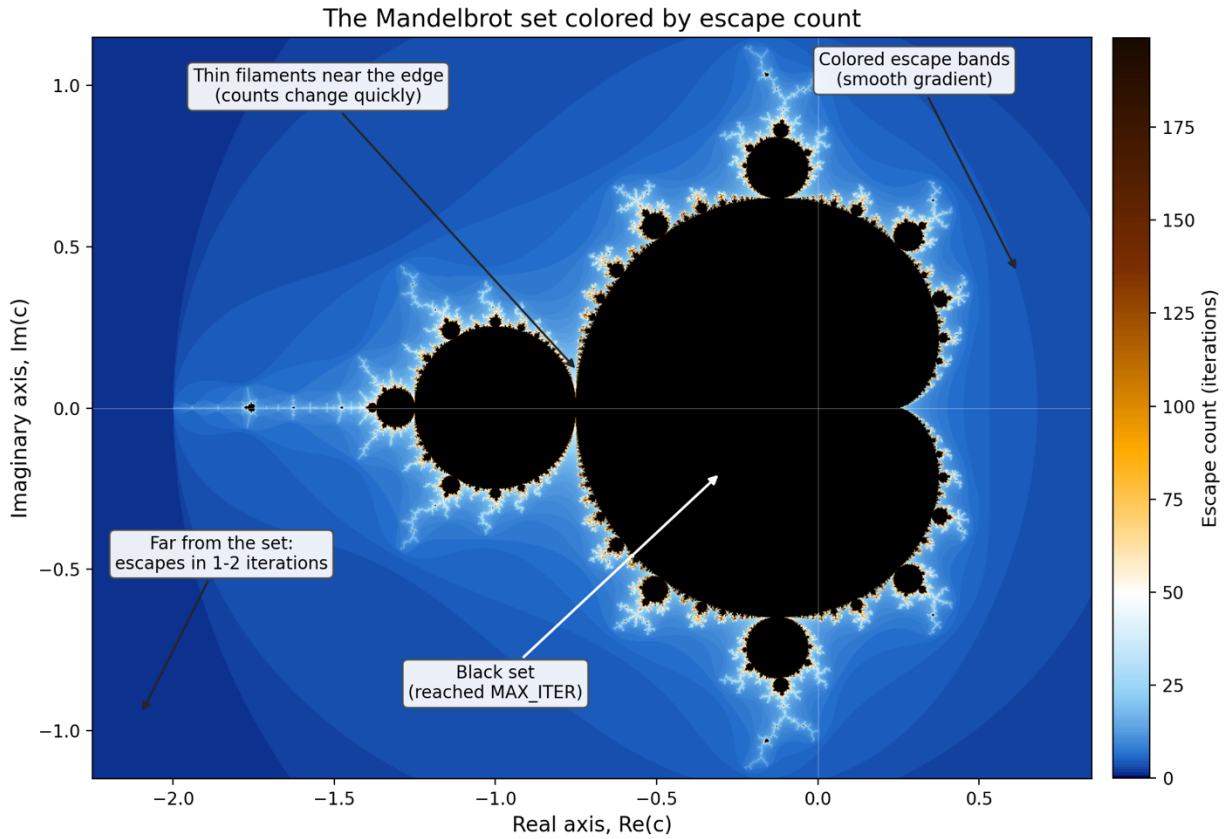
The design already computes x^2 and y^2 for the update, so the escape test costs only one addition and one comparison. Some points never escape, and the hardware cannot iterate forever, so it gives up after **MAX_ITER** iterations and treats the point as in the set.

2.4 Turning Escape Counts into a Picture

For each pixel, the engine records its **escape count**: the number of iterations before the point escaped. Points in the set reach MAX_ITER and are painted **black**. That black shape *is* the Mandelbrot set. Points outside the set are colored by how quickly they escaped (e.g. the number of iterations). Points far from the set escape after one or two iterations, and points near the edge can take a hundred or more.

The colors come from a **color table**, often called a **palette**: a list of colors numbered from 0 upward, where the escape count picks the entry. Neighboring entries are similar colors, so the palette forms a smooth gradient. In this lab it runs from deep blue, through light blue and white, to orange and near black. Points with similar escape counts get similar colors, which is why the picture shows smooth **bands** around the set. Where the escape count changes quickly from one pixel to the next, the colors change quickly too, which makes the thin filaments near

the edge stand out. The colors themselves are arbitrary: a different palette gives a different-looking picture of exactly the same escape counts.

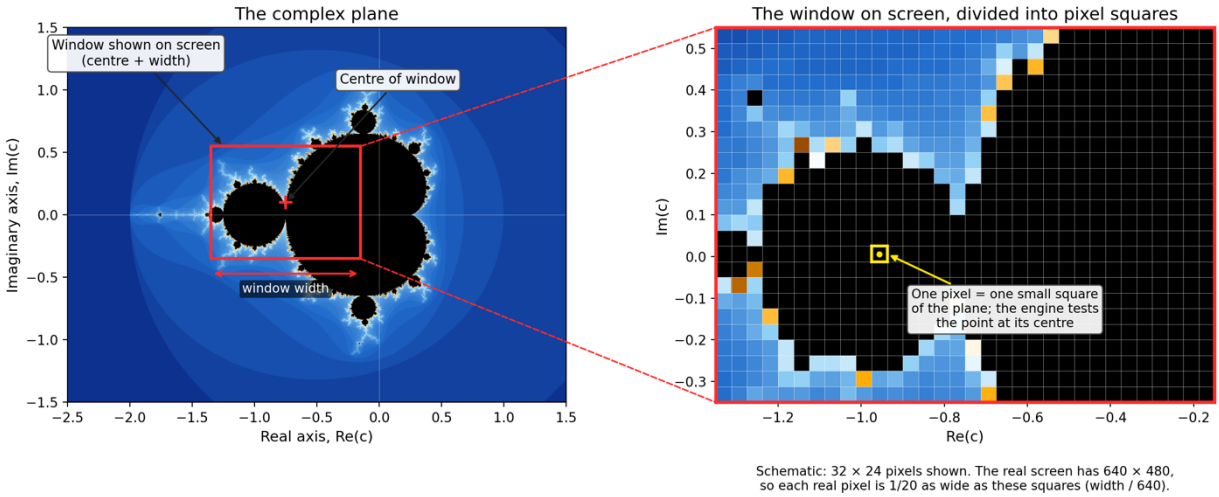


2.5 The Screen as a Window: Panning and Zooming

The plane goes on forever, but the Mandelbrot set only occupies a small part of it, about 4 units wide (from about −2.5 to +1.5 left to right). The screen shows a rectangular **window** onto the plane, like a map app on a phone showing part of a map. Two things describe the window:

- **Where it is:** the point on the plane that the window is centered on.
- **How big it is:** how much plane the window covers, from its left edge to its right edge.

The screen always has 640 × 480 pixels, so each pixel stands for one small square of the plane, and the engine tests the point at that square. For example, if the window is 4 units wide, each of the 640 pixels across is 4 / 640 = 0.00625 units wide. Moving one pixel to the right moves 0.00625 units to the right on the plane.



Zooming in makes the window **smaller** while keeping its center in the same place. The screen still has 640 pixels across, but they now cover less of the plane, so each pixel stands for a smaller square. The result is the same as zooming in on a map: everything on screen gets bigger, and finer detail becomes visible. Because the Mandelbrot set has detail at every scale, zooming into its edge keeps revealing new spirals and filaments, no matter how far you go.

Zooming out makes the window **larger**. Each pixel covers more of the plane, everything on screen gets smaller, and more of the set fits on the screen.

Panning (moving left, right, up or down) slides the window across the plane without changing its size. Moving the window to the right brings new parts of the plane in from the right edge of the screen, so the picture appears to slide to the left, just like dragging a map.

In this lab, the zoom runs continuously, a little each frame, and a switch picks whether it zooms in or out. The push buttons pan the window. The window only changes in the short gap between frames, never while a frame is being drawn, so the picture never tears.

Deep zooms make each pixel's square extremely small, so the design needs very precise numbers.

3. Project Setup

You will use **ModelSim** to simulate and verify your design, and **Altera Quartus Prime** to compile, synthesize and program it on the DE25-Standard FPGA development board.

The lab uses the same directory structure as the earlier labs:

```
Lab 4/
├── vhdl/  – Contains the VHDL source files for the design.
├── sim/   – Contains the ModelSim command file and simulation output.
└── syn/   – Contains the Quartus Prime project, constraints and PLL IP.
```

The design files are:

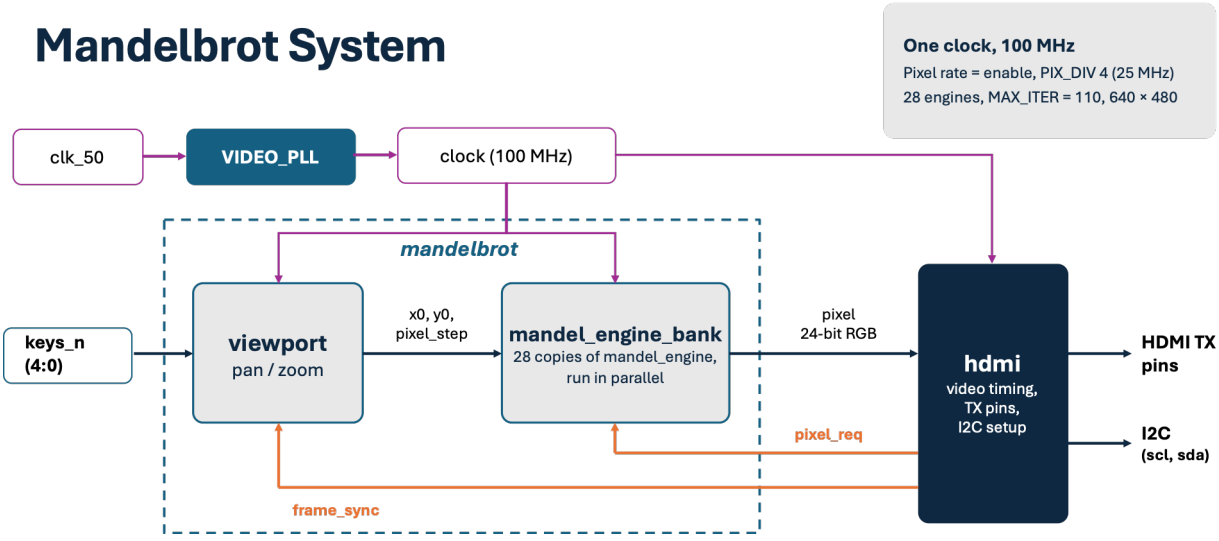
File	Status	Description
mandel_pkg.vhd	Provided	Fixed-point constants, ESCAPE_SQ, and the get_color lookup table
mandel_engine.vhd	**TODO**	One Mandelbrot engine: iterates one point and outputs its color
mandel_engine_bank.vhd	**TODO**	Walks the screen and runs N_ENGINES engines in parallel
viewport.vhd	Provided	Tracks x0, y0 and pixel_step; pans and zooms once per frame
mandelbrot.vhd	**TODO**	Connects the viewport to the engine bank
hdmi.vhd	Provided	640 × 480 video timing, HDMI transmitter pins and I2C setup
mandelbrot_system.vhd	Provided	Board-level top: PLL, reset, renderer and HDMI output
mandelbrot_tb.vhd	Provided	Testbench to simulates mandelbrot and writes frames as BMP images
syn/VIDEO_PLL.ip	Provided	Quartus PLL IP: 50 MHz in, 100 MHz out

*NOTE: Throughout this assignment, please refer to the **DE25-Standard Development Kit User Manual** and the **DE25-Standard Getting Started Guide**, which contain more details on the board setup and pinouts. The datasheet for the board's HDMI transmitter chip, the **Analog Devices ADV7513** (ADV7513.pdf), is also provided. It describes the video input pins and the I2C registers that the hdmi module configures.*

4. System Design

4.1 Architecture

The design has three main parts: the **viewport** decides where on the plane the screen is looking, the **engine bank** computes the color of every pixel, and the **HDMI** module sends the pixels to the monitor. There is **no frame buffer**. Each pixel is computed just before the display needs it and is sent straight to the monitor.



- **viewport** (provided): keeps track of the window as three values: **x0** and **y0**, the point on the plane at the top-left pixel of the screen, and **pixel_step**, the width of the piece of

the plane each pixel covers. Once per frame, during the gap between frames, it makes pixel_step smaller or larger to zoom, and it moves x0 and y0 to pan when a key is held.

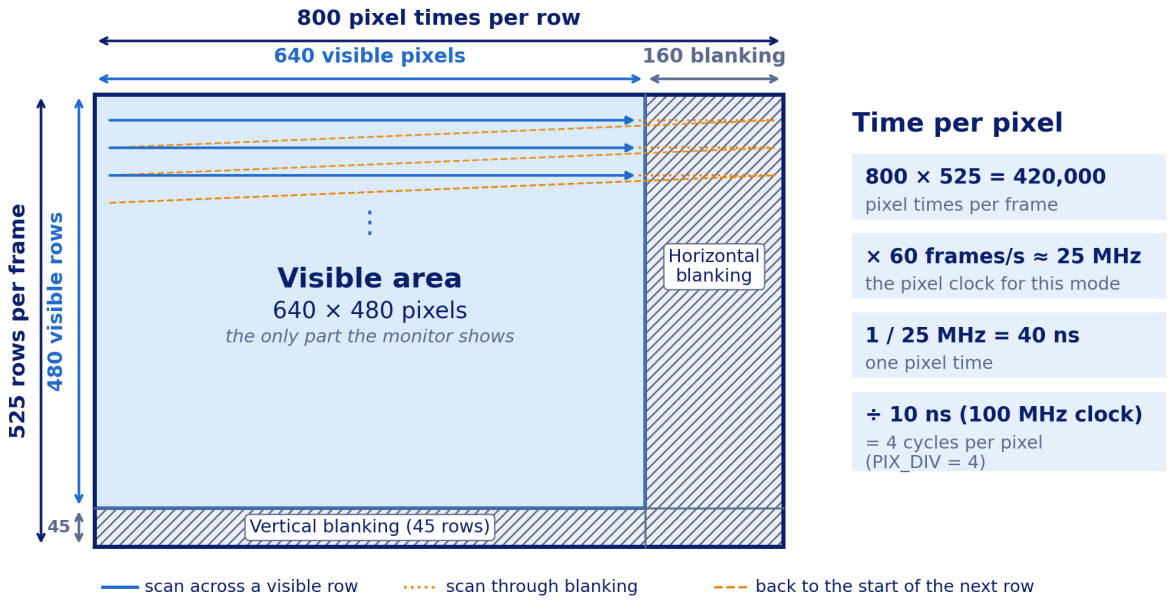
- **mandel_engine_bank:** walks the screen in raster order (left to right, top to bottom), computes the point c for each pixel, and hands it to the next engine in turn. When the display asks for a pixel, it returns that engine's finished color.
- **mandel_engine:** runs the Mandelbrot rule for one point c, one iteration per clock cycle, and outputs the color for its escape count.
- **hdmi** (provided): generates the 640 × 480, 60 Hz video timing. It asks for one pixel at a time with pixel_req, signals the gap between frames with frame_sync, and drives the board's ADV7513 HDMI transmitter, which it configures over I2C (see the provided ADV7513 datasheet).
- **video_pll:** generates a 100MHz clock from the 50MHz system clock to drive all the components.

4.2 The Parallel Computing Engines

The time available per pixel is set by the monitor, not by the design:

1. **The video signal sends 60 frames a second.** An HDMI monitor only accepts standard video modes, combinations of resolution and refresh rate. This lab uses **640 × 480 pixels at 60 Hz**, the one mode every HDMI monitor is required to support. HDMI is a continuous stream, so the FPGA must send a complete frame every 1/60 of a second, even when the picture has not changed.
2. **Every frame includes invisible pixels.** The monitor scans each frame like reading a page: one row at a time, left to right, top to bottom. The video standard adds a short invisible stretch at the end of every row and every frame, called **blanking**, which the monitor uses to get ready for the next row or frame. So each row is really 800 pixel times long, of which 640 are visible, and each frame is 525 rows, of which 480 are visible.
3. **That fixes the pixel rate.** 800 × 525 pixel times per frame × 60 frames per second ≈ 25 million pixel times per second, the standard **25 MHz pixel clock** for this mode. One pixel time is 1 / 25 MHz = **40 ns**.
4. **At the 100 MHz system clock, that is 4 cycles per pixel.** One cycle is 10 ns, so 40ns is 4 cycles. This is PIX_DIV = 4 in the code.

The design has no frame buffer, so a pixel's color must be ready within its 40 ns, every frame. A design with a frame buffer in memory would not have this limit: the engines could take as long as they needed, and the HDMI side would keep re-sending the stored picture at 60 Hz. This design trades that memory for compute speed.



One engine performs one iteration per clock cycle, so a pixel takes up to **MAX_ITER cycles**. With only 4 cycles per pixel, a single engine would be nearly 30 times too slow. The iterations cannot be sped up, because each one needs the result of the one before.

Instead, the Mandelbrot engine instantiates N engine banks to work on many pixels at once. Engine 0 takes pixel 0, engine 1 takes pixel 1, and so on, wrapping back to engine 0 after the last engine. A new pixel is handed out every 4 cycles, so each engine gets N_ENGINES × 4 cycles before its next pixel is due:

$$MAX_ITER + 2 \leq N_ENGINES \times PIX_DIV$$
$$110 + 2 \leq 28 \times 4 = 112$$

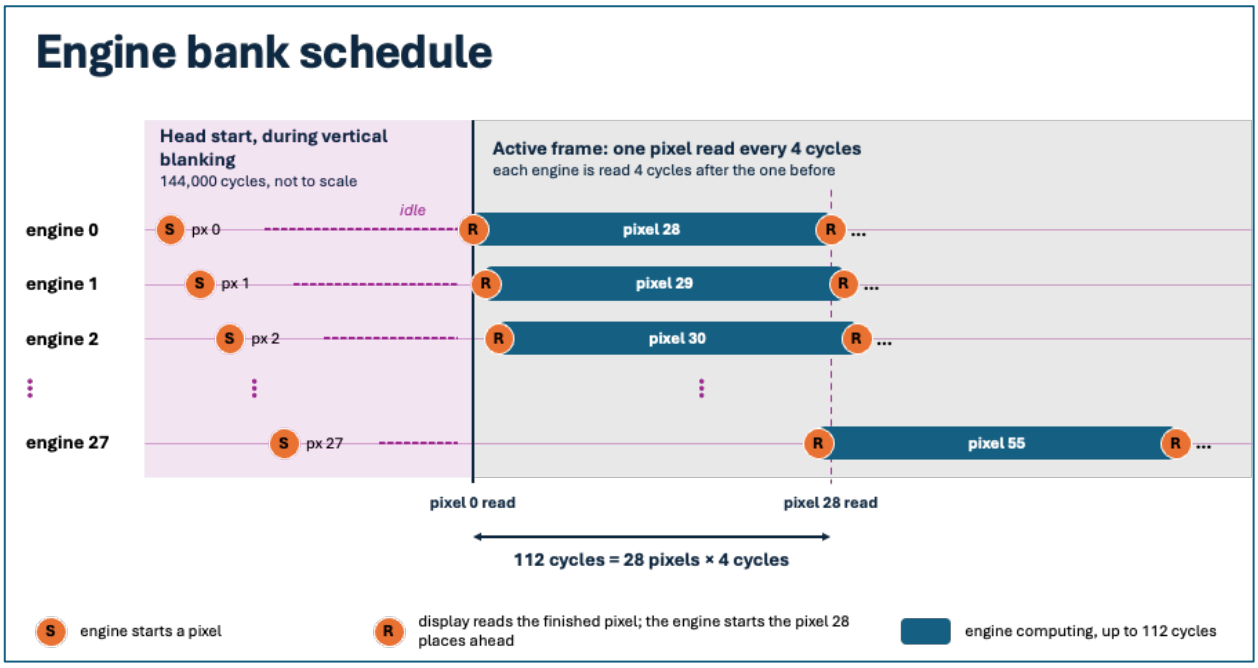
The two extra cycles are one to start the engine and one for its output register.

We derive **28 engines** for the bank from the limitation in the number of the FPGA's **DSP blocks**, the dedicated hardware multipliers. Each engine needs three 36-bit multiplies, which use 6 DSP blocks. The device has 188, so at most 31 engines fit, and 28 leaves the fitter some room.

110 iterations is then the most that fit in the 112 cycles each engine gets. It is not a property of the Mandelbrot set. More iterations would show more detail near the edge of the set, and the only ways to get them are more engines or a faster clock.

Each pixel takes up to 112 cycles to compute, so pixel 0 must be started at least 112 cycles before the display reads it. The bank does this in the **vertical blanking**, the gap between frames when the display reads nothing (45 invisible rows × 800 × 4 = 144,000 cycles). It starts the first 28 pixels on the 28 engines there, so they are all finished when the frame begins.

After that, the pipeline keeps itself full. Whenever the display reads a pixel from an engine, the same engine starts the pixel 28 places ahead. That pixel is read 28 pixel periods later, which is exactly 28 × 4 = 112 cycles, so it is always finished in time. The head start only has to happen once per frame.



4.3 Clocking and the Video PLL

The board provides a **50 MHz** oscillator, but the design needs two other rates:

- **The pixel rate.** The HDMI standard defines the clock rate for each video mode. For 640 × 480 at 60 Hz, it is 25.175 MHz, and 25 MHz is close enough for monitors to lock onto. The display must receive pixels at exactly this rate, or the monitor cannot display the picture.
- **The compute rate.** The engines need as many cycles per pixel as possible. At 50 MHz there would only be 2 cycles per pixel, or 56 iterations across 28 engines.

A **PLL** (phase-locked loop) is a circuit that synthesizes a new clock frequency from a reference clock by multiplying and dividing it. The provided VIDEO_PLL IP takes the 50 MHz oscillator and produces a **100 MHz** system clock. Despite its name, this clock drives the **entire design**, the engines as well as the video output. Only the board oscillator runs at 50 MHz, and it is used for nothing but the PLL's input. The hdmi module divides that by PIX_DIV = 4 to get the 25 MHz pixel rate, and generates the HDMI transmitter clock (hdmi_tx_clk) from the same divider.

The 25 MHz pixel rate is a **clock enable** on the 100 MHz clock, not a separate clock. The whole design runs on a single clock, so no signals ever need to cross between clock domains. The 100MHz rate gives the engines 4 cycles per pixel, which allows 110 iterations. It is also close to

the limit: one iteration takes about 9 ns of logic, so a much faster clock would no longer meet timing.

The PLL is created with the Quartus IP Catalog (IOPLL) and stored in syn/VIDEO_PLL.ip, which is already part of the project. It is instantiated in mandelbrot_system.vhd. The PLL is kept out of the mandelbrot module so that the testbench can drive the system clock directly.

5. Design Implementation

In this section, you will complete the **TODO** sections in three files. Work from the bottom of the hierarchy up: the engine, then the bank, then mandelbrot.

5.1 Fixed-Point Numbers and numeric_std

Fixed-point numbers

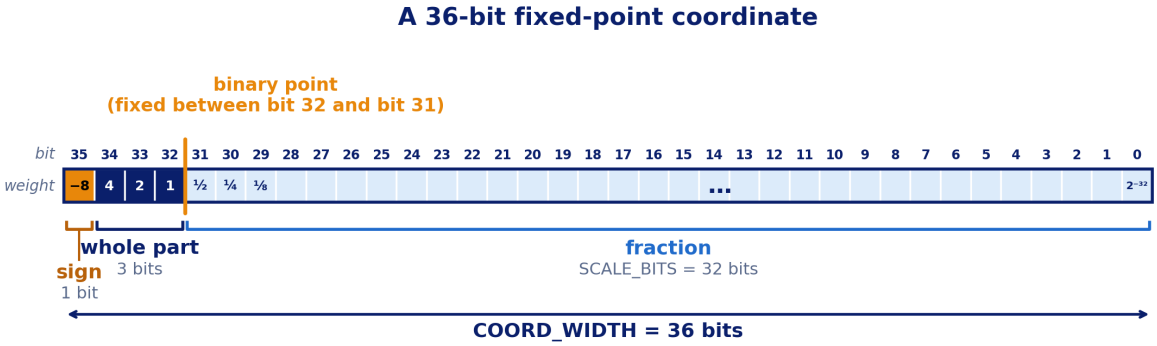
The adders and multipliers in an FPGA only work with **whole numbers** (integers). But the Mandelbrot set lives in a region only about 4 units wide, so the design needs to work with fractions like 0.75 or −1.0625. **Fixed-point** numbers solve this by storing a fraction as a whole number of very small units as a power of 2, because they are easy for binary hardware. This design counts in units of 2^{-32}, about 0.00000000023, so a coordinate is stored as the whole number

$$\text{stored integer: } N = \text{value} \times 2^{32}$$

$$\text{value read back} = \frac{N}{2^{32}}$$

In binary, this means the **low 32 bits are the fraction** and the bits above them are the whole-number part, as if there were a binary point fixed between bit 32 and bit 31. That fixed position is where the name *fixed-point* comes from.

Each coordinate is COORD_WIDTH = 36 bits wide:



The top 4 bits hold the sign and the whole-number part, for a range of about −8 to +8. That is plenty, because the Mandelbrot calculation never needs values larger than this before a point escapes. For example:

Value	Stored integer N
1.0	2^{32} = 4,294,967,296
0.5	2^{31} = 2,147,483,648
−0.5	-2^{31}
4.0 (ESCAPE_SQ)	4×2^{32}

Addition and subtraction work directly, because both values are counted in the same steps:

$$(a \times 2^{32}) + (b \times 2^{32}) = (a + b) \times 2^{32}$$

Multiplication does not work directly, as the size of its result is the sum of the left hand side and right hand side:

$$(a \times 2^{32}) \times (b \times 2^{32}) = (a \times b) \times 2^{64}$$

Dividing by 2^{32} means dropping the bottom 32 bits. After every multiply, you keep a **slice** of the product that starts at bit SCALE_BITS. Multiplying two 36-bit numbers gives a 72-bit product, and bits 67 down to 32 are the 36-bit result back on the correct scale. The bottom 32 bits, the part smaller than one step, are discarded.

The engine uses two different tools to control the width of its numbers, and each has its own job:

- **A bit slice makes a value narrower.** Slicing bits 67 down to 32 out of the product rescales it back to 36 bits. Dropping the bottom 32 bits is a shift right by 32, which divides by 2^{32}, and it works for negative two's-complement values too. Dropping the top 4 bits is safe as long as the result fits in the ±8 range, because those bits are then just copies of the sign bit, and bit 67 becomes the new sign bit.
- **resize makes a value wider.** It fills in the new upper bits correctly (see the resize section below), so the value does not change. The engine uses it in the escape test: the two squares are each resized one bit wider before they are added, so that their sum has room for the carry and cannot overflow.

A slice is only correct if the result really fits in the bits you keep. The Mandelbrot update always fits, because it only runs while the point is within distance 2 of the origin. The escape test does not: on the iteration where a point escapes, $x^2 + y^2$ can reach about 36, so that test slices GUARD_BITS more bits out of each product, then uses resize to widen both before adding.

signed and unsigned

Every signal in this lab is a std_logic_vector, which is just a group of wires with no numeric meaning. To do arithmetic, you **cast** it to signed or unsigned from the ieee.numeric_std package. The cast tells VHDL how to interpret the top bit:

Type	Top bit means	Range of an *n*-bit value	Use it for
signed	the sign	-2^{n-1} to $2^{n-1} - 1$	x, y, cx, cy: coordinates can be negative
unsigned	another power of 2	0 to $2^n - 1$	the escape count, and $x^2 + y^2$, which are never negative

The same bits mean different things under each cast: "1111" is −1 as signed but 15 as unsigned. No tool warns you if you choose wrong. Casting a coordinate to unsigned turns the left half of the picture inside out.

The numeric_std rules you will need:

- **Multiply:** the result is as wide as both operands combined. Two 36-bit operands give a 72-bit result, which can never overflow.
- **Add and subtract:** the result is as wide as the **wider** operand. It does not grow, so if a sum might need an extra bit, widen the operands first with resize(value, new_width).
- **Converting back:** wrap the result in std_logic_vector(…) before assigning it to a signal.

resize

resize(value, new_width) changes the number of bits in a signed or unsigned value. When widening, it keeps the value the same by filling the new upper bits correctly for the type:

Type	New upper bits	Example: 4 bits widened to 6
unsigned	zeros (**zero extension**)	1100 (12) → 001100 (12)
signed	copies of the sign bit (**sign extension**)	1100 (−4) → 111100 (−4)

Use it whenever a result needs more bits than its operands. For example, before adding two numbers whose sum might overflow, resize both operands one bit wider, so the sum has room for its carry. The same bits widened the wrong way change their value: 1100 sign-extended as unsigned would read as 60, not 12. The pattern is to cast in the expression, do the arithmetic, and convert back to std_logic_vector, for example:

```
total <= std_logic_vector(signed(a) + signed(b));
```

5.2 Implement the Mandelbrot Engine

Complete **mandel_engine.vhd**. One engine computes the color of one pixel. It receives a point c on cx and cy with a one-clock start pulse, runs the Mandelbrot rule one iteration per clock cycle, and then raises ready with the pixel's color on pixel.

Each step below says which part of the Mandelbrot algorithm it implements.

Step 1: Declare the engine's state. Add signals in the architecture for everything the engine must remember from one clock to the next:

- **z**, the running point: its real part x and imaginary part y. These are 36-bit coordinates, used as signed.
- **c**, the pixel being computed: registered copies of cx and cy. The bank moves on to the next pixel after the start pulse, so the engine keeps its own copy.
- **An escaped flag:** set once z has passed the escape radius.
- **The escape count:** how many iterations happened before the point escaped. This is ITER_WIDTH bits, used as unsigned.
- **A clock counter, from 0 to MAX_ITER:** how many cycles have passed since start.

Step 2: Reset. Clear everything to zero, except the clock counter. Set the clock counter to MAX_ITER, which means "finished", so that after reset the engine is idle instead of computing a point nobody asked for.

Step 3: Start a new point (z = 0). When start = '1', save cx and cy, set x and y to zero, and clear the escaped flag, the escape count and the clock counter. This is the "start z at the origin" part of the algorithm.

Step 4: One iteration per clock cycle. On every clock while the clock counter is less than MAX_ITER, perform one iteration of $z \leftarrow z^2 + c$:

- **a. Compute the three products x·x, y·y and x·y.** These are the only multiplies in the algorithm: new x needs x^2 and y^2, new y needs xy, and the escape test reuses x^2 and y^2. Use **signed** multiplies at the full 72-bit width.
- **b. Rescale the products.** Each product is 2^{32} too big. Keep bits SCALE_BITS + COORD_WIDTH − 1 down to SCALE_BITS of each one to get x^2, y^2 and xy as 36-bit coordinates.
- **c. Compute $x^2 + y^2$ for the escape test.** On the iteration where z first escapes, $x^2 + y^2$ can be as large as about 36, which does not fit in the ±8 range of a coordinate and would wrap around. For this test only, keep a slice that is GUARD_BITS (4) bits taller. Treat each square as **unsigned**, because a square is never negative, and resize both to COORD_WIDTH + GUARD_BITS + 1 bits before adding, so that the sum cannot overflow.
- **d. Escape test, or update.** Only if the point has not escaped yet:
  - If $x^2 + y^2 \geq$ ESCAPE_SQ (the value 4), the point has **escaped**. Set the escaped flag. From now on, x, y and the escape count stop changing.
  - Otherwise, apply the Mandelbrot rule with **signed** arithmetic, **new $x = x^2 - y^2 + cx$** and **new $y = 2xy + cy$**, and add 1 to the escape count with **unsigned** arithmetic. To double xy, add it to itself.
- **e. Advance the clock counter**, on every iteration, whether or not the point has escaped.

Step 5: Report the result. On every clock, set pixel to **get_color(escape count, MAX_ITER)**, which turns the count into a 24-bit RGB color. Set ready to '0' when start arrives, and to '1' once the clock counter reaches MAX_ITER.

How get_color turns the count into a color. get_color in mandel_pkg.vhd is provided. It is a **ROM** (read-only memory): a constant array of colors, PALETTE, indexed by a number. In VHDL, a constant array read with a signal as its index is how you describe a ROM, and synthesis builds it as a lookup table. PALETTE holds PALETTE_SIZE = 128 entries, and each entry is a 24-bit RGB color (8 bits each of red, green and blue).

get_color uses the escape count to choose the entry:

- **Escape count = MAX_ITER:** the point never escaped, so it is in the set, and get_color returns **black**.
- **Otherwise:** the escape count becomes the index into PALETTE. On the board, MAX_ITER = 110 and the count needs 7 bits, the same as the 128-entry table, so escape count *n* simply selects entry *n*. When MAX_ITER is smaller, as in the testbench (62, a 6-bit count), get_color shifts the count left to spread it across the whole table, so the full gradient is still used. Shifting by a constant amount costs no logic: it only changes which wires connect to the table's address.

Each engine has its own copy of this ROM, so it outputs a finished color rather than a count, and the bank only has to choose which engine's color to pass to the display.

Why the engine does not stop early. The escape count stops when the point escapes, but the clock counter always runs to MAX_ITER. Every pixel therefore takes exactly the same number of cycles, so the bank can schedule all 28 engines just by counting, without waiting on each one. Freezing z after it escapes also keeps it from growing until it overflows.

5.3 Implement the Engine Bank

Complete mandel_engine_bank.vhd. The walk across the screen (control_process) and the choice of which engine to start (dispatch_process) are provided. Read through them to see how the bank turns screen position into points on the plane.

The viewport describes the window with three values: **x0** and **y0**, the point on the plane at the top-left pixel, and **pixel_step**, the width of one pixel's square. The pixel in column *col* and row *row* is then the point

$$c_x = x_0 + col \times pixel_step$$
$$c_y = y_0 + row \times pixel_step$$

The bank builds this with adders instead of multipliers: cx starts at x0 and adds pixel_step for each pixel across, and cy adds pixel_step for each new row. Zooming in makes pixel_step smaller, and panning changes x0 and y0.

Instantiate N_ENGINES copies of mandel_engine, numbered 0 to N_ENGINES – 1 using the for-generate statement. Connect each engine *i* as follows:

- MAX_ITER generic: the bank's MAX_ITER.
- clock and reset: shared by all engines.
- cx and cy: connect **every** engine to the registered point cx_r and cy_r. All engines see the same point, but only the engine whose turn it is receives a start pulse.
- start, ready and pixel: connect to element *i* of engine_start, engine_ready and engine_pixel arrays.

The provided code then reads engine_pixel(slot) for the display. The same slot counter picks both the engine that is read and the engine that is restarted, so pixels always come back in the order they were handed out.

5.4 Implement the Mandelbrot Top Level

Complete mandelbrot.vhd. This is a structural design with no processes. You will need to instantiate and connect the viewport and mandel_engine_bank components.

The mandelbrot_system.vhd, which connects the HDMI, PLL and Mandelbrot together is already complete, and no work is required. However, you will need to adjust certain constants later on to evaluate different configurations when testing the system.

6. Simulation

Before programming the FPGA, verify the design in ModelSim. The testbench simulates mandelbrot and generates the display's pixel_req and frame_sync signals itself, instead of simulating the HDMI module. It captures every pixel of selected frames and writes them to **BMP image files** in the sim directory, so you can check your design by looking at the picture.

6.1 Testbench Setup

The provided mandelbrot_tb uses smaller settings than the board, to keep the simulation time reasonable:

Setting	Board	Testbench	Why
N_ENGINES	28	16	Fewer engines to simulate; the scheduling works the same at any count
MAX_ITER	110	62	Iterations are what cost simulation time
PIX_DIV	4	4	Not changed: same cycles per pixel as the board
ZOOM_NUM	255	15	Zooms faster, so a few frames show visible movement

The testbench starts at a point on the edge of the set known as *Seahorse Valley* (−0.7436 + 0.1318i) and writes **8 images**, image_1.bmp to image_8.bmp, 12 frames apart. Each image should show the view zoomed further into the same point.

Each drawn frame is about 2.5 million cycles, so the full run takes several minutes. While you are debugging, lower N_IMAGES in the testbench to write fewer images.

How the Testbench Writes Images

Checking a 640 × 480 picture pixel by pixel in a waveform viewer is impossible, so the testbench saves each captured frame as an image file. Read the write_bmp procedure in mandelbrot_tb.vhd to see how.

VHDL's standard textio package writes lines of text. An image file is **binary**: a sequence of raw bytes. To write bytes, the testbench declares a file type whose elements are characters, and writes each byte as the character with that code:

```
type char_file_t is file of character;
...
write(f, character'val(byte_value));    -- writes one byte, 0 to 255
```

This is the same technique the Lab 3 testbench used to write a WAV file. The image is written in the **BMP** format, chosen because it is one of the simplest image formats: no compression, just a header followed by the pixels.

- **Header (54 bytes):** the letters BM, the file size, the image width and height, and the number of bits per pixel (24). Multi-byte numbers are written **little-endian**, lowest byte first, by the helper procedure put_le.
- **Pixel data:** 3 bytes per pixel, in the order **blue, green, red**, which is the reverse of the pixel signal's red-green-blue layout. Rows are stored **bottom row first**, and each row is padded with zero bytes to a multiple of 4 bytes. (A 640-pixel row is 1,920 bytes, which already is a multiple of 4.)

While the simulation runs, the testbench stores each requested pixel in an array. At the end of a captured frame, it calls write_bmp to write the whole array to image_N.bmp.

6.2 The ModelSim Command File

The provided sim/sim.do compiles the files in bottom-up order, starting with mandel_pkg, adds waveform groups for the testbench, the viewport, the engine bank and engine 0, and runs the simulation. You will need to adjust the labels of your instances below accordingly.

```
vlib work

vcom -2008 ../vhdl/mandel_pkg.vhd
vcom -2008 ../vhdl/mandel_engine.vhd
vcom -2008 ../vhdl/mandel_engine_bank.vhd
vcom -2008 ../vhdl/viewport.vhd
vcom -2008 ../vhdl/mandelbrot.vhd
vcom -2008 ../vhdl/mandelbrot_tb.vhd

vsim -voptargs=+acc mandelbrot_tb

add wave -group "TESTBENCH"    sim:/mandelbrot_tb/*
add wave -group "MANDELBROT"   sim:/mandelbrot_tb/dut/*
add wave -group "VIEWPORT"     sim:/mandelbrot_tb/dut/u_viewport/*
add wave -group "ENGINE BANK"  sim:/mandelbrot_tb/dut/u_engine_bank/*
add wave -group "ENGINE 0"     sim:/mandelbrot_tb/dut/u_engine_bank/gen_engines(0)/u_engine/*

run -all
quit -f
```

The add wave paths only work if you used the labels. The last line, quit -f, closes ModelSim when the run finishes. Remove it if you want to inspect the waveforms afterwards.

6.3 Run the Simulation

Open ModelSim and change to the sim directory. Run the command file by entering:

```
do sim.do
```

Alternatively, the simulation can be launched directly from a terminal:

```
vsim –do sim.do
```

6.4 Verify the Results

Open the BMP images in the sim directory. Verify that:

- The images show the Mandelbrot set: a black region with colored bands around it.
- Each image is zoomed further into the same point than the one before.
- The picture is not sheared, torn or shifted sideways. A sheared picture usually means the engines are connected to the wrong slot or the pixels are coming back out of order.
- The colors are smooth bands, not noise. Noise usually means a coordinate was treated as unsigned, or a product was sliced at the wrong bits.

In the waveform viewer, check engine 0:

- After start, the counter counts from 0 to MAX_ITER and ready goes high at the end.
- x and y change on each clock until the point escapes, then stop changing.
- The escape count stops when the escaped flag goes high.

7. Synthesis

Once the design simulates correctly, implement it on the DE25-Standard.

7.1 Open the Quartus Prime Project

Open the provided project syn/mandelbrot.qpf in Quartus Prime. It is already set up with:

- Top-level entity: **mandelbrot_system**
- Device: Agilex 5, **A5ED013BB32AE4SCS**
- All the VHDL files from the vhdl directory, except the testbench
- The PLL IP, VIDEO_PLL.ip
- The timing constraints file, mandelbrot.sdc

You need to complete the timing constraints and assign the pins.

7.2 Timing Constraints

The **Synopsys Design Constraints** (.sdc) file tells the Quartus Timing Analyzer how fast each clock runs and which paths it should check. Without correct constraints, Quartus cannot tell whether your design will actually work at 100 MHz. The provided syn/mandelbrot.sdc only defines the board clock. Replace its contents with the following:

```
# Board oscillator (input to VIDEO_PLL)
create_clock -period "50MHz" [get_ports clk_50]

# VIDEO_PLL output: 100 MHz system clock
derive_pll_clocks

# Clock jitter and uncertainty
derive_clock_uncertainty

# Push buttons and switch: asynchronous, human-speed inputs
set_false_path -from [get_ports {reset_n keys_n[*]}]

# HDMI video outputs: clock and data aligned by construction in hdmi.vhd
set_false_path -to [get_ports {hdmi_tx_clk hdmi_tx_d[*] hdmi_tx_hs hdmi_tx_vs hdmi_tx_de}]

# HDMI interrupt and I2C bus: slow signals
set_false_path -from [get_ports {hdmi_tx_int i2c_sda}]
set_false_path -to   [get_ports {i2c_scl i2c_sda}]

# viewport registers change only once per frame
set viewport_regs [get_registers {*u_viewport|x0_r* *u_viewport|y0_r* *u_viewport|pixel_step_r*}]
set_multicycle_path -setup 4 -from $viewport_regs -to $viewport_regs
set_multicycle_path -hold  3 -from $viewport_regs -to $viewport_regs
```

What each command does, and why the design needs it:

- **create_clock** defines the 50 MHz clock arriving on the clk_50 pin. Every timing check starts from a defined clock.
- **derive_pll_clocks** creates the clocks generated by the PLL, using the settings stored in the PLL IP, so you do not have to work out and type in the 100 MHz output yourself. Without it, the 100 MHz system clock, which drives the entire design, would not be constrained, and Quartus would not check whether the engines meet timing.
- **derive_clock_uncertainty** adds the small timing margins caused by clock jitter, PLL jitter and clock-network variation. Without it, the timing report would be slightly optimistic.
- **set_false_path** tells the Timing Analyzer that a path does not need to meet a single-clock timing requirement. Otherwise Quartus reports violations on paths that cannot actually fail, which hides the real problems. This design uses it in three places:
    - **Push buttons and switch:** a person presses them at random times, with no relationship to the clock, so there is no timing requirement to meet.
    - **HDMI video outputs:** hdmi_tx_clk is generated by the same divider that produces the data, so the transmitter clock and data leave the FPGA together and stay aligned. That alignment is built into hdmi.vhd rather than enforced by a constraint.
    - **HDMI interrupt and I2C:** these change at tens of kilohertz, thousands of cycles apart.
- **set_multicycle_path** tells the Timing Analyzer that a path has more than one clock period to settle. The viewport updates x0, y0 and pixel_step only once per frame,

millions of cycles apart, so the zoom arithmetic between those registers does not need to finish in one 10 ns clock. -setup 4 gives it 4 clock periods. -hold 3 moves the hold check back to the original clock edge, which is required whenever the setup requirement is moved. These constraints find the registers by name, which is why the viewport instance must be labelled u_viewport.

7.3 Assign FPGA Pins

Open **Assignments → Pin Planner** and assign the following signals:

Port	FPGA Pin	I/O Standard	Description
clk_50	BM71	1.2-V	50 MHz board clock
reset_n	BM78	1.2-V	CPU reset button
keys_n[0]	BW59	1.2-V	KEY0: pan left
keys_n[1]	CA59	1.2-V	KEY1: pan right
keys_n[2]	CF71	1.2-V	KEY2: pan up
keys_n[3]	CH71	1.2-V	KEY3: pan down
keys_n[4]	BM62	1.2-V	SW0: zoom direction (0 = in, 1 = out)
hdmi_tx_clk	CH59	1.2-V	HDMI transmitter pixel clock
hdmi_tx_hs	BR109	3.3-V LVCMOS	HDMI horizontal sync
hdmi_tx_vs	BE107	3.3-V LVCMOS	HDMI vertical sync
hdmi_tx_de	BK109	3.3-V LVCMOS	HDMI data enable
hdmi_tx_d[0]	CD134	3.3-V LVCMOS	HDMI pixel data bit 0
hdmi_tx_d[1]	CD135	3.3-V LVCMOS	HDMI pixel data bit 1
hdmi_tx_d[2]	CG134	3.3-V LVCMOS	HDMI pixel data bit 2
hdmi_tx_d[3]	CG135	3.3-V LVCMOS	HDMI pixel data bit 3
hdmi_tx_d[4]	CH132	3.3-V LVCMOS	HDMI pixel data bit 4
hdmi_tx_d[5]	CF132	3.3-V LVCMOS	HDMI pixel data bit 5
hdmi_tx_d[6]	CF128	3.3-V LVCMOS	HDMI pixel data bit 6
hdmi_tx_d[7]	CK134	3.3-V LVCMOS	HDMI pixel data bit 7
hdmi_tx_d[8]	CL125	3.3-V LVCMOS	HDMI pixel data bit 8
hdmi_tx_d[9]	CF121	3.3-V LVCMOS	HDMI pixel data bit 9
hdmi_tx_d[10]	CF118	3.3-V LVCMOS	HDMI pixel data bit 10
hdmi_tx_d[11]	BU118	3.3-V LVCMOS	HDMI pixel data bit 11
hdmi_tx_d[12]	BR118	3.3-V LVCMOS	HDMI pixel data bit 12
hdmi_tx_d[13]	CA118	3.3-V LVCMOS	HDMI pixel data bit 13
hdmi_tx_d[14]	BW118	3.3-V LVCMOS	HDMI pixel data bit 14
hdmi_tx_d[15]	CL128	3.3-V LVCMOS	HDMI pixel data bit 15
hdmi_tx_d[16]	CL130	3.3-V LVCMOS	HDMI pixel data bit 16
hdmi_tx_d[17]	CK125	3.3-V LVCMOS	HDMI pixel data bit 17

Port	FPGA Pin	I/O Standard	Description
hdmi_tx_d[18]	CK128	3.3-V LVCMOS	HDMI pixel data bit 18
hdmi_tx_d[19]	BF111	3.3-V LVCMOS	HDMI pixel data bit 19
hdmi_tx_d[20]	BH109	3.3-V LVCMOS	HDMI pixel data bit 20
hdmi_tx_d[21]	BE115	3.3-V LVCMOS	HDMI pixel data bit 21
hdmi_tx_d[22]	BF115	3.3-V LVCMOS	HDMI pixel data bit 22
hdmi_tx_d[23]	BU109	3.3-V LVCMOS	HDMI pixel data bit 23
hdmi_tx_int	BE111	3.3-V LVCMOS	HDMI transmitter interrupt (active low)
i2c_scl	BF120	3.3-V LVCMOS	HDMI transmitter I2C clock
i2c_sda	BH118	3.3-V LVCMOS	HDMI transmitter I2C data

7.4 Compile and Synthesize

Run the complete Quartus Prime compilation flow. The compilation should complete without errors. Review the compilation reports and verify that:

- mandelbrot_system is the top-level entity, and all top-level ports have pin assignments and I/O standards.
- **DSP blocks:** the Fitter's resource usage summary shows about 6 DSP blocks per engine, 168 for 28 engines. Far fewer means the multipliers were optimized away, which usually means part of the engine is not connected.
- **Timing:** the Timing Analyzer shows the 100 MHz PLL output clock, and it meets timing with no setup or hold violations. If the 100 MHz clock is missing from the report, check the derive_pll_clocks line in the SDC file.
- **Constraints applied:** the multicycle constraints matched the viewport registers. If Quartus warns that get_registers matched nothing, check that the viewport instance is labelled u_viewport.

8. Hardware Setup and Testing

8.1 Connect the DE25-Standard

Make all connections with the FPGA board powered off. Connect the DE25-Standard to the computer through the board's **USB-Blaster** connection, and connect a monitor to the board's **HDMI** port.

8.2 Program and Test the System

Power on the board and the monitor, open **Tools → Programmer** in Quartus Prime, select the .sof file, and program the FPGA. After a moment the monitor should show the Mandelbrot set.

The controls are:

Control	Action
KEY0 / KEY1	Pan left / right (hold)
KEY2 / KEY3	Pan up / down (hold)
SW0	Zoom direction: 0 = zoom in, 1 = zoom out
CPU reset	Return to the starting view

The zoom always holds the center of the screen still. Left alone, it zooms into the center of the starting view, which is inside the solid black body of the set, so the screen soon turns completely black. That is correct behavior. To see detail, pan onto the **edge** of the set while the view is still wide, then let the zoom carry you in. The seam on the left side of the main body, near −0.75 on the real axis, is the easiest place to start.

Test the system:

- Verify that the full set appears at reset, centered on the screen.
- Pan in all four directions.
- Zoom in on the edge of the set and verify that new detail keeps appearing. Then flip SW0 and zoom back out.
- Press reset and verify that the view returns to the start.
- Unplug and reconnect the HDMI cable, and verify that the picture comes back.

8.3 Things to Try

mandelbrot_system.vhd has a few settings you can change and rebuild:

- **ZOOM_NUM** sets the zoom speed. A table in the file lists the time to reach each zoom level for each value.
- **CENTER_X_NUM, CENTER_Y_NUM and VIEW_WIDTH** set the starting view. The file lists two starting points with more detail than the default: *Seahorse Valley* and a *tendril*.

9. Submission

Zip your project folder, name it **CE355_Lab4_Team_[N].zip**, and submit it to Canvas along with your report. Clean out synthesis and simulation work directories before zipping, and do not include the generated .bmp files.

9.1 Report

Submit a report of **no more than 2–3 pages**, figures included. Be concise: every figure must be referenced and explained in the text, and must be legible — crop and zoom rather than pasting a full-desktop screenshot. Cover the following.

- **Simulation.** The transcript output from the simulator log window, including one or two annotated waveforms and explanation that shows how you know the design is working.

Include explanation of any test cases you ran. Include any output rendered in simulation (such as images or text).

- **Schematic.** An RTL Viewer image showing your structure and correct connections, including instantiation of key components — block RAMs, shift registers, FIFOs, PLLs, DSP blocks, IP. Note anything the tool inferred that you did not intend: a latch, LUT-based memory instead of a block RAM, a multiplier built from logic.
- **Resources.** Based on the synthesis results, determine if the resources used align with what is expected, for example: number of multipliers (DSPs), memory bits and the blocks, approximate register count, shift registers, and any other IPs. Explain any significant differences.
- **Timing.** fMAX and worst-case setup slack for each clock, the registers the critical path runs between, and why it is the slowest path. “Timing met” is not an analysis. Explain any changes you made to your design to make timing if it wasn’t met initially.
- **Optimizations.** Discuss any optimizations or techniques you implemented and its significance.
- **Hardware and attribution.** Which hardware tests passed, and note any differences between simulation and hardware.

9.2 Demonstration

You will also demo the hardware and simulation during TA office hours. Be prepared to explain how the system works, including the Mandelbrot iteration and escape test, the fixed-point multiply and why the product is sliced, where you used signed and unsigned arithmetic, why the design needs many engines and a PLL, and what each timing constraint does.